materials_25_02_2026

Хочешь — следующим шагом сделаю “авто-билд”

То есть после успешного ingest сервис сам публикует событие “build tiles” (без ручного вызова), а воркеры строят.

Скажи только: **всегда строим тайлы сразу или по флагу?**

давай по флагу. И еще ты сейчас сделал вариант когда мы храним статус в mongo. А почему не во внутренней или оперативной памяти? как лучше?

Ок — **по флагу** делаем легко: после успешного ingest, если `build_tiles=true`, публикуем задачу в Rabbit и создаём job в Mongo.

А про статусы: **в памяти можно**, но это почти всегда хуже в микросервисах, если нужна надёжность. Ниже — честное сравнение и “как лучше” под твой кейс.

---

1) Ingest “по флагу”: как сделать

1.1 Добавь флаг в endpoint ingest

В `app/api/admin_routes.py` → `ingest()` добавь query-параметр:

```
build_tiles:bool=Query(default=False,description="If true — e
nqueue tiles build job")
```

и после успешного `meta = svc.ingest(...)` добавь enqueue:

```
fromapp.api.depsimportget_tile_build_queue,get_jobs_repo
fromuuidimportuuid4

# ...
ifbuild_tiles:
job_id=str(uuid4())
payload= {
"job_id":job_id,
```

```
"uuid":meta.uuid,
"tile_size":256,# или тоже параметрами
"fmt":"webp",
"lossless":False,
"attempt":0,
"source":"ingest",
    }
jobs=get_jobs_repo()
q=get_tile_build_queue()
jobs.create(job_id=job_id,uuid=meta.uuid,payload=payload)
q.publish_build(payload)
```

Чтобы было гибко, можно добавить ещё параметры:

- `tiles_tile_size=256|512`
- `tiles_fmt=webp|png`
- `tiles_lossless=false`

Я могу прямо дать готовый кусок, если хочешь, чтобы всё было query-параметрами.

---

2) Статусы job: Mongo vs память — что лучше и почему

Вариант A — в оперативной памяти

✅ Плюсы:

- очень быстро
- простая реализация

❌ Минусы (критичные):

- **при рестарте сервиса всё исчезает**
- если запущено **несколько инстансов API/worker** — статусы “разъезжаются”
- нельзя нормально дебажить: нет истории, нет причин ошибок

- если воркер упал после ACK/в процессе — статус может остаться “running” навсегда

👉 Подходит только для:

- локальной разработки
- демо
- строго один инстанс, и “потеря статуса не страшна”

Вариант B — Mongo (или Postgres)

✅ Плюсы:

- **надёжно**: статусы переживают рестарт
- работает при **масштабировании** (несколько API/worker)
- удобно мониторить, алертить, расследовать “почему упало”
- можно хранить payload, attempt, error, timestamps

❌ Минусы:

- чуть сложнее
- чуть больше I/O (но это мелочь по сравнению со сборкой тайлов)

👉 Для твоего кейса (MinIO+Mongo уже есть, jobs важны, будут ретраи/DLQ) — **Mongo лучше**.

---

3) “Как лучше” конкретно для твоего проекта

Я бы рекомендовал так:

✅ По умолчанию: Mongo (как мы сделали)

- `queued/running/done/failed`
- `attempt`, `error`
- `created_at/updated_at`
- TTL-очистка старых job-ов (например, хранить 7 дней)

Плюс: можно сделать ручки:

- `GET /tiles/jobs/{job_id}`
- `GET /tiles/jobs?uuid=...`
- `POST /tiles/jobs/{job_id}/requeue` (если упало и хочешь повторить)

Опционально: маленький in-memory cache статусов (ускорение)

Можно дополнительно держать последние N статусов в памяти, но **источник истины** всё равно Mongo.

---

4) Важный нюанс: ACK и “потеря” задач

Очередь гарантирует доставку, но:

- если воркер ACK-нул сообщение и умер до завершения — задачу уже не переиграешь.

Поэтому твой текущий воркер правильно делает:

- ACK только после завершения *или* сам перепубликует retry/DLQ и ACK исходное.

Ещё более надёжно:

- включать `basic_nack(requeue=True)` при transient-ошибках (но там надо аккуратно, иначе бесконечный requeue).

---

5) Что предлагаю сделать следующим (без вопросов)

1. Добавить в ingest endpoint параметры:
   - `build_tiles: bool = false`
   - `tiles_tile_size: int = 256`
   - `tiles_fmt: TileFormat = "webp"`
   - `tiles_lossless: bool = false`
2. Enqueue job после ingest, если `build_tiles=true`

3. Добавить TTL индекс в Mongo jobs (например 7 дней), чтобы коллекция не росла бесконечно

Если скажешь “да”, я сразу дам точные патчи к твоему `admin_routes.py` + TTL-index для jobs repo.

Понял. У тебя всё уже разложено по двум роутерам: `app/api/admin_routes.py` (ingest/storage/metadata) и `app/api/tiles_routes.py` (tiles). Значит делаем “по флагу” **внутри `ingest()` из `admin_routes.py`**.

И ещё важный момент: у тебя сейчас в `tiles_routes.py` **/jobs/{job_id} объявлен после /{uuid}/{z}/{y}/{x}** — это почти гарантированно будет ломаться (FastAPI будет пытаться распарсить `jobs` как `uuid`, а `job_id` как `z` и т.д.). Нужно просто поднять job-роуты выше.

Ниже — конкретные правки.

---

1) Добавляем флаг в ingest и enqueue в очередь

1.1 Импорты в `app/api/admin_routes.py`

Вверху добавь:

```
fromuuidimportuuid4
fromapp.api.depsimportget_tile_build_queue,get_jobs_repo
fromapp.services.tile_build_queueimportTileBuildQueue
fromapp.repos.mongo_jobs_repoimportMongoJobsRepository
fromapp.domain.tilesimportTileFormat
```

1.2 Меняем сигнатуру `ingest()` и добавляем enqueue-логику

Заменяй твой `ingest()` на этот (я сохранил твою текущую логику + добавил параметры и блок после успешного ingest):

```
@ingest_router.post("/images/{storage}/ingest")
defingest(
storage:str,
```

```
file:UploadFile=File(...),
uuid:str|None=Query(default=None,description="Optional UUID f
or idempotency"),
on_conflict:str=Query(default="error",pattern="^(error|overwr
ite|skip)$"),

# --- NEW: флаг + параметры билда тайлов ---
build_tiles:bool=Query(default=False,description="If true — e
nqueue tiles build job"),
tiles_tile_size:int=Query(256,description="256 or 512",alias
="tiles_tile_size"),
tiles_fmt:TileFormat=Query("webp",description="webp or png",a
lias="tiles_fmt"),
tiles_lossless:bool=Query(False,alias="tiles_lossless"),

q:TileBuildQueue=Depends(get_tile_build_queue),
jobs:MongoJobsRepository=Depends(get_jobs_repo),
):
try:
svc=get_ingest_service(storage)
exceptException:
raiseHTTPException(status_code=400,detail="storage must be on
e of: fs, mem, s3")

try:
meta=svc.ingest(
uuid=uuid,
on_conflict=on_conflict,# type: ignore[arg-type]
filename=file.filename,
content_type=file.content_type,
upload_file_stream=file.file,
        )
exceptFileExistsErrorase:
raiseHTTPException(status_code=409,detail=str(e))
exceptExceptionase:
raiseHTTPException(status_code=400,detail=f"Ingest failed:
```

```
{e}")

job_info=None
ifbuild_tiles:
iftiles_tile_sizenotin (256,512):
raiseHTTPException(status_code=400,detail="tiles_tile_size mu
st be 256 or 512")
iftiles_fmtnotin ("webp","png"):
raiseHTTPException(status_code=400,detail="tiles_fmt must be
webp or png")

job_id=str(uuid4())
payload= {
"job_id":job_id,
"uuid":meta.uuid,
"tile_size":tiles_tile_size,
"fmt":tiles_fmt,
"lossless":tiles_lossless,
"attempt":0,
"source":"ingest",
        }

# статус в Mongo + публикация в Rabbit
try:
jobs.create(job_id=job_id,uuid=meta.uuid,payload=payload)
q.publish_build(payload)
job_info= {"job_id":job_id,"status":"queued"}
exceptExceptionase:
# ingestion уже успешен — просто возвращаем, что enqueue не у
далось
job_info= {"job_id":job_id,"status":"enqueue_failed","error":
str(e)}

return {
"uuid":meta.uuid,
"uri":meta.uri,
```

```
 "storage":meta.storage,
 "name":meta.name,
 "last_updated":meta.last_updated.isoformat(),
 "content_type":meta.content_type,
 "size_bytes":meta.size_bytes,
 "width":meta.width,
 "height":meta.height,
 "format":meta.format,
 "mode":meta.mode,
 "bucket":meta.bucket,
 "key":meta.key,
 "path":meta.path,
 "tiles_job":job_info,# NEW
     }
```

Теперь ты можешь грузить так:

- без тайлов:

  `POST /ingest/images/s3/ingest`

- с автоматическим билдом тайлов:

  `POST /ingest/images/s3/ingest?build_tiles=true&tiles_tile_size=256&tiles_fmt=webp&tiles_lossless=false`

---

2) ВАЖНО: почини порядок роутов в `tiles_routes.py`

Сейчас у тебя:

- `/tiles/{uuid}/{z}/{y}/{x}` стоит **раньше**, чем `/tiles/jobs/{job_id}` (и, кажется, раньше любых “нечисловых” путей).

Нужно сделать так:

1. `_cache/*`
2. `/jobs/*`
3. `/build-async`
4. `/build`

5. `/{uuid}/manifest`
6. `/{uuid}/{z}/{y}/{x}` (самый общий — внизу)

Быстрый фикс

Просто **перенеси блоки**:

- `@tiles_router.get("/jobs/{job_id}")` и `@tiles_router.post("/{uuid}/build-async")` **выше** `@tiles_router.get("/{uuid}/{z}/{y}/{x}")`.

Иначе `/tiles/jobs/123` попытается матчиться на `/{uuid}/{z}/{y}/{x}` и упадёт на конвертации `z=int("123")` или ещё раньше.

---

3) Почему статус лучше хранить в Mongo, а не в RAM

Коротко по делу:

Статус в памяти (RAM)

Подходит только если:

- один инстанс API и один воркер
- рестарты не страшны
- не важно “что было с задачей вчера”

Минусы:

- **рестарт = потеря статусов**
- несколько воркеров/API = статусы “разъезжаются”
- нет истории ошибок/попыток

Статус в Mongo (как у тебя)

Лучше для прод-режима:

- переживает рестарты
- общий источник истины для всех инстансов
- можно дебажить (ошибка, попытки, время)

- легко делать мониторинг/алерты

Для твоего кейса (Rabbit + retry/DLQ) **Mongo — правильнее**.

Если хочешь, следующим шагом я дам **TTL-индекс для jobs** (например, автоудаление job-документов через 7 дней), чтобы коллекция не раздувалась.

давай

Супер. Делаем TTL-очистку jobs в Mongo так, чтобы:

- у каждого job-документа было поле `expires_at: datetime`
- Mongo сам удалял документ, когда `expires_at` наступит
- срок хранения настраивался через `.env` / `config.py`

---

1) Добавь переменную TTL в `config.py` и `.env`

`config.py` (внутри `Settings`)

```
# сколько хранить jobs в Mongo (секунды)
MONGO_JOBS_TTL_SECONDS:int=7*24*60*60# 7 days
```

`.env`

```
MONGO_JOBS_TTL_SECONDS=604800
```

---

2) Обнови `app/repos/mongo_jobs_repo.py` — TTL index + expires_at

Заменяй файл на этот вариант (или внеси изменения):

```
from__future__importannotations
fromdatetimeimportdatetime,timedelta
fromtypingimportOptional
frompymongo.collectionimportCollection
```

```
classMongoJobsRepository:
def__init__(self,col:Collection,*,ttl_seconds:int):
self.col=col
self.ttl_seconds=int(ttl_seconds)

self.col.create_index("job_id",unique=True)
self.col.create_index("uuid")

# TTL index: Mongo удалит документ, когда expires_at <= now
# expireAfterSeconds=0 => удаляем сразу по наступлению expire
s_at
self.col.create_index("expires_at",expireAfterSeconds=0)

defcreate(self,*,job_id:str,uuid:str,payload:dict) ->None:
now=datetime.utcnow()
expires_at=now+timedelta(seconds=self.ttl_seconds)

self.col.update_one(
            {"job_id":job_id},
            {"$setOnInsert": {
"job_id":job_id,
"uuid":uuid,
"payload":payload,
"status":"queued",
"created_at":now,
"updated_at":now,
"expires_at":expires_at,# <-- важно для TTL
"error":None,
"attempt":0,
            }},
upsert=True,
        )

defset_status(self,job_id:str,*,status:str,attempt:int,error:
```

```
Optional[str]=None) ->None:
now=datetime.utcnow()
self.col.update_one(
            {"job_id":job_id},
            {"$set": {"status":status,"attempt":attempt,"erro
r":error,"updated_at":now}},
        )

defget(self,job_id:str) ->Optional[dict]:
returnself.col.find_one({"job_id":job_id}, {"_id":0})
```

> TTL удаление в Mongo работает не “секунда в секунду”: обычно уборщик проходит примерно раз в минуту — это нормально.

---

3) Обнови `get_jobs_repo()` в `app/api/deps.py`

У тебя было примерно так:

```
@lru_cache(maxsize=1)
defget_jobs_repo() ->MongoJobsRepository:
mongo=get_mongo_client()
col=mongo[settings.MONGO_DB][settings.MONGO_JOBS_COLLECTION]
returnMongoJobsRepository(col)
```

Станет так:

```
@lru_cache(maxsize=1)
defget_jobs_repo() ->MongoJobsRepository:
mongo=get_mongo_client()
col=mongo[settings.MONGO_DB][settings.MONGO_JOBS_COLLECTION]
returnMongoJobsRepository(col,ttl_seconds=settings.MONGO_JOBS
_TTL_SECONDS)
```

---

4) (Опционально, но полезно) ручка stats по jobs

Если хочешь быстро видеть сколько jobs хранится:

```
# где-нибудь в tiles_routes.py или отдельном admin router
fromapp.api.depsimportget_jobs_repo
fromapp.repos.mongo_jobs_repoimportMongoJobsRepository

@tiles_router.get("/jobs/_stats")
defjobs_stats(jobs:MongoJobsRepository=Depends(get_jobs_rep
o)):
# repo не экспонирует collection — можно добавить метод count
()
# или если хочешь — я добавлю его в repo.
return {"status":"ok","note":"add count() if you want"}
```

(Если скажешь — добавлю `count()` и `find_by_uuid()`.)

Если хочешь, следующим шагом я бы сделал ещё одно важное улучшение:

идемпотентность билда тайлов (чтобы повторные сообщения из очереди не пересобирали тайлы зря, а проверяли наличие manifest и/или “build version”).

добавь - 4) (Опционально, но полезно) ручка stats по jobs

Если хочешь быстро видеть сколько jobs хранится:

# где-нибудь в tiles_routes.py или отдельном admin router
from app.api.deps import get_jobs_repo
from app.repos.mongo_jobs_repo import MongoJobsRepository

@tiles_router.get("/jobs/_stats")
def jobs_stats(jobs: MongoJobsRepository = Depends(get_jobs_repo)):
    # repo не экспонирует collection — можно добавить метод count()
    # или если хочешь — я добавлю его в repo.
    return {"status": "ok", "note": "add count() if you want"}

(Если скажешь — добавлю count() и find_by_uuid().)

Ок, добавляю **полноценную stats-ручку** + методы `count()` и `find_by_uuid()` в `MongoJobsRepository`.

---

1) Обнови `app/repos/mongo_jobs_repo.py`

Добавь в класс два метода (и лучше сразу весь блок вставить в конец класса):

```
defcount(self) ->int:
returnint(self.col.count_documents({}))

defcount_by_status(self) ->dict:
# простая агрегация по status
pipeline= [
            {"$group": {"_id":"$status","n": {"$sum":1}}},
        ]
res= {"queued":0,"running":0,"done":0,"failed":0}
forrowinself.col.aggregate(pipeline):
res[str(row["_id"])]=int(row["n"])
returnres

deffind_by_uuid(self,uuid:str,*,limit:int=50) ->list[dict]:
cur=self.col.find({"uuid":uuid}, {"_id":0}).sort("created_a
t",-1).limit(int(limit))
returnlist(cur)
```

> Если у тебя в документе есть `created_at`, сортировка ок (мы её создаём при `create()`).

---

2) Добавь endpoint stats в `app/api/tiles_routes.py`

ВАЖНО про порядок роутов

Размести эти ручки **выше** `@tiles_router.get("/{uuid}/{z}/{y}/{x}")`, иначе маршрут с параметрами может перехватывать `jobs/...`.

Добавь импорты (если ещё нет):

```
fromapp.api.depsimportget_jobs_repo
fromapp.repos.mongo_jobs_repoimportMongoJobsRepository
```

И добавь ручки:

```
@tiles_router.get("/jobs/_stats")
defjobs_stats(jobs:MongoJobsRepository=Depends(get_jobs_rep
o)):
return {
"status":"ok",
"total":jobs.count(),
"by_status":jobs.count_by_status(),
    }


@tiles_router.get("/jobs/by-uuid/{uuid}")
defjobs_by_uuid(uuid:str,jobs:MongoJobsRepository=Depends(get
_jobs_repo),limit:int=Query(50,ge=1,le=500)):
return {
"status":"ok",
"uuid":uuid,
"items":jobs.find_by_uuid(uuid,limit=limit),
    }
```

---

3) Что ты получишь

- `GET /tiles/jobs/_stats` → сколько всего job-ов и разбивка по статусам
- `GET /tiles/jobs/by-uuid/<uuid>?limit=50` → последние job-ы по конкретному изображению

Если хочешь, я ещё добавлю **ручку “requeue failed job”**:

`POST /tiles/jobs/{job_id}/requeue` — возьмёт payload из Mongo и заново отправит в Rabbit (удобно при ручном восстановлении после DLQ).

объясни мне еще раз подробно про RabbitMQ и как он у меня тут используется и реализован

Конечно. Давай разложу по полочкам, **что такое RabbitMQ** и **как именно он у тебя используется в текущей схеме**, которую мы собрали.

Что такое RabbitMQ (на практике)

RabbitMQ — это **брокер сообщений** (message broker). Он делает 3 вещи:

1. **принимает сообщения** (jobs/события) от отправителей (producers)
2. **кладёт их в очередь** (queue) и хранит (может на диске, если durable/persistent)
3. **отдаёт сообщения потребителям** (consumers/workers), которые выполняют работу

Он нужен, когда работу нельзя/неудобно делать сразу в HTTP-запросе:

- долго
- тяжело
- нужны ретраи
- надо распараллелить
- надо “не потерять” задачу при рестарте

Как он используется у тебя (в твоём проекте)

У тебя есть сценарий: **“после загрузки оригинала построить пирамиду тайлов”**.

Было (сейчас у тебя есть синхронный build)

`POST /tiles/{uuid}/build`

→ API начинает читать оригинал, ресайзить, резать, заливать тайлы

→ клиент ждёт (может долго)

Стало (асинхронно через очередь)

1. Клиент загружает оригинал:
   `POST /ingest/images/{storage}/ingest`
2. Если передан флаг `build_tiles=true`, API **не строит тайлы**, а:

- создаёт запись job в Mongo (`status=queued`)
- отправляет сообщение в RabbitMQ очередь `tile.build`
- отвечает клиенту быстро: `job_id`

1. Отдельный процесс **tile-worker** слушает очередь `tile.build` и строит тайлы в фоне.

Кто есть кто в твоей схеме

Producer (отправитель)

Это твой FastAPI API (например `ingest()` или `POST /tiles/{uuid}/build-async`).

Его роль:

- сформировать payload задачи
- отправить сообщение в Rabbit

Пример payload:

```
{
  "job_id":"....",
  "uuid":"....",
  "tile_size":256,
  "fmt":"webp",
  "lossless":false,
  "attempt":0
}
```

Queue (очередь) в RabbitMQ

У нас 3 очереди:

1. **Main queue**: `tile.build`

   Сюда попадают новые задачи

2. **Retry queue**: `tile.build.retry`

   Сюда попадает задача, если воркер упал, но мы хотим попробовать ещё раз.

   Эта очередь работает как “отложенная повторная попытка”.

3. **DLQ**: `tile.build.dlq`

   Если задача не смогла выполниться после N попыток, мы кладём её сюда (для ручного разбора).

Consumer / Worker (исполнитель)

Это отдельный контейнер/процесс `tile-worker`:

- он подписан на `tile.build`
- получает сообщение
- выполняет build:
  - открывает оригинал через `OriginalImageService` (Mongo → S3/FS/Mem)
  - спуливает в temp файл (чтобы Pillow не держал 250MB в памяти)
  - вызывает `TilePyramidBuilder.build(...)` → сохраняет тайлы в MinIO (или fs)
- обновляет статус job в Mongo

Как устроена гарантия “не потерять задачу”

В Rabbit есть понятия:

durable queue

Очередь создаётся как durable — Rabbit хранит её “как сущность” даже после рестарта.

persistent message (delivery_mode=2)

Сообщение помечается persistent — Rabbit старается писать его на диск.

Это значит:

- если Rabbit перезапустился, задача не исчезнет (в большинстве нормальных случаев)

ACK / NACK — что это и зачем

Rabbit не удаляет сообщение из очереди навсегда, пока consumer не скажет:

✅ `ACK` — “я задачу сделал, можно удалить сообщение”

Если consumer умер до ACK — Rabbit считает, что задача не выполнена, и может выдать её другому воркеру.

Как сделано у тебя

У тебя логика такая:

- если build **успешный** → `ACK`
- если build **ошибка** → мы:
  - увеличиваем `attempt`
  - публикуем сообщение либо в retry, либо в DLQ
  - и **ACK** исходного сообщения (потому что мы его “переложили” сами)

Это нормальная схема.

Как работает retry очередь (tile.build.retry)

Мы используем хитрость:

- retry queue настроена с `x-dead-letter-routing-key = tile.build`
- при публикации сообщения в retry мы задаём `expiration = delay_ms`

Что происходит:

1. сообщение лежит в `tile.build.retry` и “ждёт” TTL
2. TTL истёк → Rabbit сам “мертвит” сообщение
3. из-за DLX-настроек оно автоматически отправляется обратно в `tile.build`

4. воркер пробует ещё раз

Это называется delayed retry через TTL + DLX.

Что такое DLQ в этой схеме (tile.build.dlq)

Если `attempt > MAX_RETRIES`:

- воркер отправляет payload в `tile.build.dlq`

Там задачи просто лежат, чтобы ты мог:

- посмотреть что за uuid
- посмотреть ошибку в Mongo job record
- починить и requeue вручную

Зачем Mongo для статусов jobs

Rabbit отвечает на вопрос:

> “гарантированно доставить задачу воркеру”

Но Rabbit *не хранит удобно*:

- статус выполнения
- ошибки
- прогресс
- результаты

Поэтому Mongo jobs — это “табличка” состояния:

- queued / running / done / failed
- attempt
- error
- timestamps
- payload

Это позволяет:

- клиенту спрашивать `GET /tiles/jobs/{job_id}`
- тебе дебажить и мониторить

Как “по флагу” это работает в твоём API

В `POST /ingest/.../ingest` ты добавляешь `build_tiles=true`

Тогда после успешной загрузки:

- создаётся job
- публикуется сообщение в Rabbit
- клиент получает `{ tiles_job: { job_id, status } }`

Если флаг false:

- всё работает по-старому, без очереди.

Итог: что конкретно у тебя получилось

Ты получил архитектуру:

- **FastAPI API** — быстро принимает запросы, публикует job
- **RabbitMQ** — хранит задания и отдаёт воркерам
- **tile-worker** — отдельно масштабируемый исполнитель тяжёлой операции build
- **Mongo jobs** — трекает статус, ошибки, retries
- **DLQ** — склад проблемных задач

Если хочешь, могу ещё нарисовать “жизненный цикл сообщения” прям по шагам (на примере ошибки MinIO и 5 ретраев) и показать, где что происходит.

объясни еще раз подробно как у меня реализован DLQ

Ок, объясняю DLQ именно **в твоей реализации**, по шагам, максимально предметно.

У тебя DLQ (“dead letter queue”) реализован **логикой в воркере** + **отдельной очередью Rabbit**.

Какие очереди у тебя есть

1. **Main queue**: `tile.build`

   сюда приходят обычные задачи на билд тайлов

2. **Retry queue**: `tile.build.retry`

   сюда воркер перекладывает задачу, если она упала, но ещё можно пробовать

3. **DLQ**: `tile.build.dlq`

   сюда воркер кладёт задачу, если она упала слишком много раз (превысила лимит ретраев)

Что именно считается “плохим сообщением” у тебя

Сообщение (задача) считается “мёртвым”, если:

- в воркере обработка упала с exception
- и `attempt` после увеличения стал **больше** `TILE_BUILD_MAX_RETRIES`

То есть DLQ включается **после N неудачных попыток**.

Какие поля позволяют DLQ работать

В payload сообщения у тебя есть поле:

- `attempt` — номер попытки

Пример payload:

```
{
  "job_id":"...",
  "uuid":"...",
  "tile_size":256,
  "fmt":"webp",
```

```
  "lossless":false,
  "attempt":3
}
```

Воркер при ошибке делает:

- `attempt_next = attempt + 1`
- если `attempt_next <= MAX_RETRIES` → retry
- иначе → DLQ

Как воркер отправляет сообщение в DLQ

В `tile_worker.py` у тебя есть функция (в нашей версии) типа:

- `_publish_dlq(ch, payload)`

Она делает **обычный** `basic_publish` в очередь `tile.build.dlq`:

- routing_key = `RABBIT_DLQ`
- delivery_mode=2 (persistent)
- content_type=application/json

Важно: это **прямое перекидывание в DLQ**, а не “автоматическая DLQ политика Rabbit”.

Что происходит при ошибке — полный цикл

Представим: build падает (битый файл / MinIO недоступен / Pillow exception).

1) Воркер получил сообщение из `tile.build`

Rabbit отдал сообщение воркеру.

2) Воркер начал обработку

- `jobs.set_status(..., status="running")`

3) Случилась ошибка

В блоке `except Exception as e`:

- увеличиваем `attempt`:
  - было `attempt=2`
  - стало `attempt=3`
- пишем статус в Mongo:
  - `jobs.set_status(... status="failed", attempt=3, error="...")`

4) Ветка retry или DLQ

Если `attempt <= MAX_RETRIES`

то воркер делает:

- публикует сообщение в `tile.build.retry`
- ставит TTL через `expiration=delay_ms`
- и **ACK** исходное сообщение из `tile.build`

После TTL оно автоматом вернётся обратно в `tile.build`.

Если `attempt > MAX_RETRIES`

то воркер делает:

- публикует payload в `tile.build.dlq`
- и **ACK** исходное сообщение из `tile.build`

То есть:

✅ исходное сообщение удалено (ACK)

✅ в DLQ лежит “последняя версия” сообщения (с увеличенным attempt)

Почему в твоей схеме DLQ не “автоматическая”, а “ручная”

В RabbitMQ есть классический механизм DLQ через:

- `x-dead-letter-exchange`

- `x-dead-letter-routing-key`
- ограничения/политики (requeue, reject, max-length и т.д.)

Но у тебя DLQ сделан проще и контролируемее:

- ты сам решаешь, когда сообщение умерло
- сам публикуешь его в DLQ
- сам определяешь retry delay
- сам ведёшь attempt-счётчик

Это надёжно и предсказуемо.

Что в итоге лежит в DLQ

В DLQ лежит JSON payload задачи, например:

```
{
  "job_id":"....",
  "uuid":"....",
  "tile_size":256,
  "fmt":"webp",
  "lossless":false,
  "attempt":6,
  "source":"ingest"
}
```

И отдельно в Mongo по `job_id` лежит:

- `status: "failed"`
- `attempt: 6`
- `error: "... причина ..."`
- timestamps

Что ты можешь делать с DLQ задачами

Обычно 3 сценария:

1. **разобраться и забыть** (битый файл, невалидный uuid)
2. **починить причину** и **перезапустить** (requeue)
3. **пометить как permanently failed** (и не трогать)

Практическая фича, которую реально стоит добавить:

- `POST /tiles/jobs/{job_id}/requeue`
  - берём payload из Mongo
  - ставим `attempt=0`
  - публикуем заново в `tile.build`

Самое важное отличие DLQ от retry у тебя

- retry — временная “передышка”, задача вернётся в main
- DLQ — задача больше **не вернётся автоматически**, только вручную

Если хочешь, я могу:

- добавить ручку `GET /tiles/dlq/peek` (читать N последних сообщений из DLQ — но это сложнее, Rabbit не даёт “просто посмотреть” без consume)
- или сделать лучше: воркер при отправке в DLQ ещё пишет в Mongo поле `dead_lettered=true` и `dead_lettered_at`, чтобы видеть такие задачи без чтения очереди.

где в коде показано Какие очереди у тебя есть?

Отличный вопрос 👌

Сейчас покажу **где именно в твоём коде объявлены очереди** и откуда берётся понимание, что их три.

📍 Где в коде определены очереди

Очереди объявляются в **воркере** в файле:

```
app/workers/tile_worker.py
```

Ключевая функция:

```
def_declare_queues(ch:pika.channel.Channel) ->None:
# Main queue
ch.queue_declare(queue=settings.RABBIT_QUEUE,durable=True)

# Retry queue
ch.queue_declare(
queue=settings.RABBIT_RETRY_QUEUE,
durable=True,
arguments={
"x-dead-letter-exchange":"",
"x-dead-letter-routing-key":settings.RABBIT_QUEUE,
        },
    )

# DLQ
ch.queue_declare(queue=settings.RABBIT_DLQ,durable=True)
```

Вот здесь 👆 и создаются 3 очереди.

🧠 Откуда берутся их названия

Названия не захардкожены — они приходят из `settings`.

В `config.py`:

```
RABBIT_QUEUE:str="tile.build"
RABBIT_RETRY_QUEUE:str="tile.build.retry"
RABBIT_DLQ:str="tile.build.dlq"
```

И в `.env`:

```
RABBIT_QUEUE=tile.build
RABBIT_RETRY_QUEUE=tile.build.retry
RABBIT_DLQ=tile.build.dlq
```

То есть реально создаются очереди:

```
tile.build
tile.build.retry
tile.build.dlq
```

📦 Где используется main queue

В воркере:

```
ch.basic_consume(
queue=settings.RABBIT_QUEUE,
on_message_callback=handle_message
)
```

Это значит:

→ воркер слушает `tile.build`

🔁 Где используется retry queue

В блоке обработки ошибки:

```
_publish_retry(ch,payload,settings.TILE_BUILD_RETRY_DELAY_MS)
```

А внутри `_publish_retry`:

```
ch.basic_publish(
exchange="",
routing_key=settings.RABBIT_RETRY_QUEUE,
```

```
    ...
)
```

То есть сообщение публикуется в:

```
tile.build.retry
```

☠️ Где используется DLQ

В случае превышения числа попыток:

```
_publish_dlq(ch,payload)
```

А внутри:

```
ch.basic_publish(
exchange="",
routing_key=settings.RABBIT_DLQ,
    ...
)
```

То есть сообщение отправляется в:

```
tile.build.dlq
```

📊 Как это можно увидеть вживую

Если зайдёшь в Rabbit UI:

```
http://localhost:15672
login: guest
password: guest
```

В разделе **Queues** ты увидишь:

- tile.build
- tile.build.retry
- tile.build.dlq

🔥 Важный момент

Очереди создаются не в API, а именно в воркере при старте:

```
_declare_queues(ch)
```

Поэтому если:

- ты запустишь только API,
- но не запустишь worker,

очередей может не быть создано.

💡 Резюме

Очереди определяются в 3 местах:

Где	Что
`config.py`	Имена очередей
`_declare_queues()`	Создание очередей
`basic_publish()`	Отправка сообщений
`basic_consume()`	Чтение из main queue

Если хочешь, могу ещё нарисовать точную схему движения сообщения между этими тремя очередями.

а почему мы делаем воркеры отдельным файлом и запускаем их отдельно? не будет ли более правильно запускать воркеры вместе с сервисом?

Потому что “воркер” и “API-сервис” — это **две разные роли**. Их можно запускать вместе, но в большинстве прод-схем это хуже по управляемости и надёжности. Разложу по-честному, с плюсами/минусами и тем, как правильно именно для твоего кейса.

Почему воркер отдельным процессом (как мы сделали)

1) Разная нагрузка и разные требования

- **API**: должен отвечать быстро, держать много соединений, быть предсказуемым по latency.
- **Воркер**: CPU/RAM-heavy (Pillow resize/crop/encode), может крутиться минутами, может иногда падать.

Если их смешать в одном процессе, тяжёлая сборка тайлов:

- начнёт отбирать CPU у API
- увеличит время ответов
- может привести к таймаутам / 502 у клиентов

2) Масштабирование отдельно

Когда нагрузка растёт, тебе часто нужно:

- 2 API-инстанса (для запросов)
- 10 воркеров (для задач)

Если воркер “внутри API” — масштабировать становится неудобно: ты вынужден поднимать лишние API, чтобы получить больше воркеров.

3) Разные режимы перезапуска

Воркер может упасть из-за одной битой картинки / OOM.

Если он отдельный:

- Docker перезапустит воркер
- API продолжит работать

Если он внутри API:

- упал воркер → упал весь сервис → клиенты страдают

4) Нормальная эксплуатация

Отдельный контейнер/процесс воркера:

- отдельные логи
- отдельные лимиты CPU/RAM
- отдельный healthcheck
- можно отключать/обновлять без простоя API

Можно ли запускать “вместе с сервисом”?

Да, можно. Но важно уточнить: **вместе — это как?**

Вариант A: “в одном контейнере, но разными процессами”

Например, через `supervisord` или shell entrypoint:

- один процесс: `uvicorn main:app`
- второй процесс: `python -m app.workers.tile_worker`

✅ Плюсы:

- один образ/контейнер
- проще деплой “на коленке”

❌ Минусы:

- в Docker это считается анти-паттерном: контейнер = один главный процесс
- сложнее управлять рестартами (что рестартить? оба? только воркер?)
- нельзя масштабировать API и воркер отдельно

Вариант B: “в том же uvicorn процессе”

Тут становится совсем плохо:

- воркер блокирует event loop (или потоки), если не вынести в отдельный executor
- непредсказуемая производительность
- конкуренция за GIL/CPU
- хуже дебажить

“Правильно” ли держать воркеры отдельно?

Для твоего кейса — **да, правильнее**.

Ты делаешь:

- загрузку оригиналов
- обработку изображений (тайлы)
- MinIO и Mongo
- потенциально большие файлы

Это типичная схема:

- API → очередь → воркеры

Когда имеет смысл “вместе” (в одном контейнере)

Только если это:

- маленький проект, один сервер
- не нужна высокая надёжность
- ты хочешь минимум сервисов
- задачи редкие

Но даже тогда лучше “вместе” делать как **два процесса** (а не в одном потоке/loop).

Что я бы рекомендовал тебе (идеальный компромисс)

✅ **Один и тот же Docker image** для API и воркера

✅ **Два разных сервиса в docker-compose** (как мы сделали):

- `api` — uvicorn
- `tile-worker` — python worker

Это удобно:

- общий код
- общий образ
- но разное назначение и масштабирование

Прямой ответ на твой вопрос

> “Не будет ли более правильно запускать воркеры вместе с сервисом?”

Нет, обычно правильнее запускать отдельно, потому что:

- безопаснее (воркер не валит API)
- быстрее масштабируется
- проще контролировать ресурсы
- проще эксплуатация и логи

Если хочешь, я могу:

1. показать вариант **одного Dockerfile** и двух команд (api/worker)
2. добавить healthchecks для rabbit/mongo/minio/worker
3. сделать “автоскейл” воркеров простым числом `replicas` (или просто `docker compose up --scale tile-worker=5`)

сделай - показать вариант одного Dockerfile и двух команд (api/worker)

добавить healthchecks для rabbit/mongo/minio/worker

Думал на протяжении 22s

Ниже — **один Dockerfile** (общий для API и worker), **две команды запуска** (api/worker), и **healthchecks** для `rabbit/mongo/minio/api/worker`.

---

1) Один Dockerfile для обоих процессов

`Dockerfile`:

```
FROM python:3.11-slim

WORKDIR /app

# системные зависимости (Pillow/WebP + curl для healthcheck A
PI)
RUN apt-get update && apt-get install -y --no-install-recomme
nds \
    curl \
    libjpeg62-turbo \
    zlib1g \
    libwebp7 \
  && rm -rf /var/lib/apt/lists/*

# зависимости python
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# код
COPY . .

# по умолчанию пусть будет API (в compose мы переопределим co
mmand для worker)
EXPOSE28000
CMD ["python","main.py"]
```

> Если у тебя `requirements.txt` ещё не содержит `pika`, добавь: `pika`.

---

2) Две команды (API / Worker) из одного образа

Запуск API

```
docker compose up--build api
```

Запуск worker

```
docker compose up--build tile-worker
```

Масштабирование воркеров

```
docker compose up--build--scaletile-worker=5
```

---

3) Healthcheck для worker (python-модуль)

Создай файл: `app/workers/healthcheck.py`

```
from__future__importannotations

importsys
importsocket
importpika

fromconfigimportsettings


def_check_tcp(host:str,port:int,timeout:float=2.0) ->None:
withsocket.create_connection((host,port),timeout=timeout):
return


defmain() ->None:
# 1) TCP до Rabbit (быстро)
```

```
try:
# из amqp-url вытащим host/port через pika
params=pika.URLParameters(settings.RABBIT_URL)
host=params.hostor"rabbitmq"
port=params.portor5672
_check_tcp(host,port,timeout=2.0)
exceptExceptionase:
print(f"[worker-health] rabbit tcp failed:{e}")
sys.exit(1)

# 2) Попробовать открыть соединение и объявить main queue (чу
ть глубже проверка)
try:
conn=pika.BlockingConnection(pika.URLParameters(settings.RABB
IT_URL))
ch=conn.channel()
ch.queue_declare(queue=settings.RABBIT_QUEUE,durable=True)
conn.close()
exceptExceptionase:
print(f"[worker-health] rabbit protocol failed:{e}")
sys.exit(1)

print("[worker-health] ok")
sys.exit(0)


if__name__=="__main__":
main()
```

4) docker-compose.yml с healthchecks (rabbit/mongo/minio/api/worker)

Пример полного compose (можешь встроить в свой):

```
services:
  mongo:
    image: mongo:7
    ports:
      -"27018:27017"
    volumes:
      - mongo_data:/data/db
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL","mongosh --quiet --eval 'db.adminCom
mand(\"ping\").ok' | grep 1"]
      interval: 10s
      timeout: 5s
      retries: 10

  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports:
      -"29000:9000"
      -"29001:9001"
    volumes:
      - minio_data:/data
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL","curl -fsS http://localhost:9000/min
io/health/ready > /dev/null"]
      interval: 10s
      timeout: 5s
      retries: 10

  rabbitmq:
```

```
    image: rabbitmq:3-management
    ports:
      -"5672:5672"
      -"15672:15672"
    networks: [backend]
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: ["CMD-SHELL","rabbitmq-diagnostics -q ping"]
      interval: 10s
      timeout: 5s
      retries: 10

  api:
    build: .
    command: ["python","main.py"]
    ports:
      -"28000:28000"
    networks: [backend]
    environment:
      MONGO_URL: mongodb://mongo:27017
      MINIO_ENDPOINT: minio:9000
      RABBIT_URL: amqp://guest:guest@rabbitmq:5672/

# остальное как у тебя
      MINIO_ACCESS_KEY: minioadmin
      MINIO_SECRET_KEY: minioadmin
      MINIO_SECURE:"false"
      MINIO_BUCKET: palleon-track
    depends_on:
      mongo:
        condition: service_healthy
      minio:
        condition: service_healthy
      rabbitmq:
```

```
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL","curl -fsS http://localhost:28000/he
alth > /dev/null"]
      interval: 10s
      timeout: 5s
      retries: 10

  tile-worker:
    build: .
    command: ["python","-m","app.workers.tile_worker"]
    networks: [backend]
    environment:
      MONGO_URL: mongodb://mongo:27017
      MINIO_ENDPOINT: minio:9000
      RABBIT_URL: amqp://guest:guest@rabbitmq:5672/

      RABBIT_QUEUE: tile.build
      RABBIT_RETRY_QUEUE: tile.build.retry
      RABBIT_DLQ: tile.build.dlq

      TILE_BUILD_MAX_RETRIES:"5"
      TILE_BUILD_RETRY_DELAY_MS:"10000"

      MINIO_ACCESS_KEY: minioadmin
      MINIO_SECRET_KEY: minioadmin
      MINIO_SECURE:"false"
      MINIO_BUCKET: palleon-track

      TILES_BACKEND: s3
      TILES_BUCKET: palleon-track
    depends_on:
      mongo:
        condition: service_healthy
      minio:
        condition: service_healthy
```

```
      rabbitmq:
        condition: service_healthy
    restart: unless-stopped
    healthcheck:
      test: ["CMD","python","-m","app.workers.healthcheck"]
      interval: 15s
      timeout: 5s
      retries: 10

volumes:
  mongo_data:
  minio_data:

networks:
  backend:
    driver: bridge
```

Почему так “правильнее”

- **Один образ** → один код/зависимости, проще CI/CD.
- **Два сервиса** (api/worker) → можно отдельно перезапускать/масштабировать/лимитировать ресурсы.
- **healthchecks + depends_on condition: service_healthy** → воркер не стартует, пока Rabbit/Mongo/MinIO не готовы.

Если хочешь — добавлю ещё `resource limits` (CPU/RAM) для worker, чтобы он не душил API.

у меня в коде нигде ничего не импортируется из tile_worker.py в основной код. как тогда у меня используются воркеры?

Это нормально — **воркер и API у тебя не “импортируют” друг друга**. Они **используются как отдельные процессы**, которые общаются через RabbitMQ (и через общую базу/хранилище), а не через Python-import.

Ключевая идея

- **API** “производит” задачи → публикует сообщения в RabbitMQ
- **tile-worker** “потребляет” задачи → слушает очередь и выполняет работу
- Связь между ними — **сообщение в очереди**, а не вызов функции через import

---

Как именно воркер “подключается” к системе

1) Его запускает Docker (или ты вручную)

В `docker-compose.yml` ты делал:

```
tile-worker:
  command: python -m app.workers.tile_worker
```

То есть при старте контейнера выполняется модуль:

```
python-m app.workers.tile_worker
```

Python запускает файл `app/workers/tile_worker.py` как отдельную программу.

Это **отдельный процесс**, параллельный API.

Где в коде происходит “сцепка” API ↔ воркер

В API сцепка — это `publish` в очередь

Например, в твоём `tiles_routes.py` уже есть:

```
q.publish_build(payload)
```

или в `ingest()` по флагу мы добавляли `q.publish_build(payload)`.

Это значит:

- API отправил JSON-пейлоад в очередь `tile.build`

В воркере сцепка — это `consume` из очереди

В `tile_worker.py` будет что-то типа:

```
ch.basic_consume(queue=settings.RABBIT_QUEUE,on_message_callb
ack=handle_message)
ch.start_consuming()
```

Это значит:

- воркер слушает очередь `tile.build`
- когда появляется сообщение — Rabbit отдает его воркеру
- `handle_message()` вызывается и билдит тайлы

Почему import не нужен и даже вреден

Если бы API “импортировал” воркер и пытался вызвать его функцию — это было бы **не очередь**, а просто вызов кода “внутри одного процесса”.

Очередь нужна как раз чтобы:

- выполнять тяжёлое отдельно
- масштабировать воркеры отдельно
- делать ретраи/DLQ
- не блокировать HTTP

Как выглядит поток данных в твоей реализации

1. `POST /ingest/.../ingest?build_tiles=true`
2. API:
    - записал оригинал (MinIO/fs/mem)
    - записал мету в Mongo
    - **создал job в Mongo**
    - **publish в Rabbit** (`tile.build`)
3. Rabbit хранит сообщение

4. tile-worker:
   - `consume tile.build`
   - открывает оригинал по uuid через `OriginalImageService` (мета из Mongo + чтение из MinIO)
   - строит тайлы (`TilePyramidBuilder`)
   - пишет статус job (`done/failed`) в Mongo
5. Клиент проверяет `GET /tiles/jobs/{job_id}`

Как убедиться, что воркер реально работает

1. Подними всё:

```
docker compose up--build
```

1. Зайди в Rabbit UI:

- `http://localhost:15672` (guest/guest)
- Queues → `tile.build`
- если воркер не запущен, там будет расти `Ready`
- если воркер работает — `Ready` будет падать, а `Unacked` кратковременно появляться

1. Дерни `POST /tiles/{uuid}/build-async`

- появится job в Mongo
- в очереди появится сообщение
- воркер его заберет

Самое важное

> Воркеры “используются” не через импорт, а через **запуск отдельного процесса** и **подписку на очередь**.

Если хочешь — кинь свой текущий `docker-compose` (где есть api сервис) и я покажу, где именно добавить `tile-worker` и как запустить `--scale tile-worker=3`, чтобы оно у тебя реально крутилось параллельно.

вот у меня есть маршрут который добавляет изображение. и есть маршрут который делит на тайлы добавленное изображение. Я думал что когда у нас например закончил выполнение маршрут 1 ставится задача в очередь , потом из этой очереди свободный воркер берет эту задачу и выполняет маршрут 2. Я не вижу чтобы у меня в воркере использовались какие -то маршруты и что то выполнялось

Ты мыслишь абсолютно правильно по **бизнес-логике**, но есть важный нюанс:

воркер обычно НЕ вызывает “маршрут 2” (HTTP endpoint).

Он выполняет **ту же самую внутреннюю бизнес-функцию**, которая стоит за этим маршрутом.

Почему так делают:

- HTTP-эндпоинт — это “транспорт” (вход/выход, валидация, HTTP-ошибки)
- воркеру транспорт не нужен — ему нужна **чистая логика**: `TilePyramidBuilder.build(...)`
- если воркер будет ходить в HTTP к своему же API — это:
  - лишняя сеть, лишний сериалайзинг
  - сложнее отлаживать
  - риск циклов/таймаутов
  - дублирование auth/headers/etc.

То есть у тебя сейчас реализовано так:

- маршрут 2 (`POST /tiles/{uuid}/build`) внутри себя делает:
  - открыть оригинал через `OriginalImageService`
  - спулить во временный файл
  - `TilePyramidBuilder.build(...)`
- воркер делает **то же самое**, но напрямую, без HTTP.

Именно поэтому ты “не видишь”, что воркер использует маршруты — он использует **сервисы**, которые маршруты тоже используют.

Где конкретно в воркере должна быть “логика маршрута 2”

Смотри на твой `build_tiles()`:

```
meta,loc,stream=svc.open_original(uuid)
# ... spool to tmp
withImage.open(tmp_path)asim:
manifest=builder.build(...)
```

Воркер должен делать то же самое.

Если ты хочешь прям 1-в-1, вынеси это в отдельную функцию/сервис, чтобы и API, и воркер вызывали одно и то же место.

Самый правильный рефактор: вынести “маршрут 2” в сервис BuildTilesService

1) Создай `app/services/tiles_build_service.py`

```
from__future__importannotations

importos
importtempfile
fromPILimportImage

fromapp.services.original_image_serviceimportOriginalImageSer
vice
fromapp.services.tile_pyramid_builderimportTilePyramidBuilder
fromapp.domain.tilesimportTileFormat
```

```
defstream_minio(resp):
try:
whileTrue:
chunk=resp.read(1024*1024)
ifnotchunk:
break
yieldchunk
finally:
try:
resp.close()
resp.release_conn()
exceptException:
pass


classTilesBuildService:
def__init__(self,original:OriginalImageService,builder:TilePy
ramidBuilder):
self.original=original
self.builder=builder

defbuild_for_uuid(self,*,uuid:str,tile_size:int,fmt:TileForma
t,lossless:bool) ->dict:
try:
meta,loc,stream=self.original.open_original(uuid)
exceptFileNotFoundError:
raiseFileNotFoundError("Original not found")

tmp_path=None
try:
withtempfile.NamedTemporaryFile(delete=False,suffix=".img")as
tmp:
tmp_path=tmp.name
ifmeta.storage=="s3":
forchunkinstream_minio(stream):
```

```
tmp.write(chunk)
else:
whileTrue:
chunk=stream.read(1024*1024)
ifnotchunk:
break
tmp.write(chunk)
tmp.flush()

withImage.open(tmp_path)asim:
im.load()
manifest=self.builder.build(uuid=uuid,image=im,tile_size=tile
_size,fmt=fmt,lossless=lossless)

return {
"uuid":manifest.uuid,
"tile_size":manifest.tile_size,
"format":manifest.format,
"lossless":manifest.lossless,
"levels": {str(z):vars(li)forz,liinmanifest.levels.items()},
            }

finally:
try:
stream.close()
getattr(stream,"release_conn",lambda:None)()
exceptException:
pass
iftmp_path:
try:
os.remove(tmp_path)
exceptException:
pass
```

2) Добавь deps для этого сервиса (`app/api/deps.py`)

```
fromapp.services.tiles_build_serviceimportTilesBuildService

@lru_cache(maxsize=1)
defget_tiles_build_service() ->TilesBuildService:
returnTilesBuildService(original=get_original_service(),build
er=get_tile_builder())
```

3) Маршрут 2 начинает вызывать сервис (и API, и воркер будут одинаковыми)

В `tiles_routes.py`:

```
fromapp.api.depsimportget_tiles_build_service
fromapp.services.tiles_build_serviceimportTilesBuildService

@tiles_router.post("/{uuid}/build")
defbuild_tiles(
uuid:str,
tile_size:int=Query(256),
fmt:TileFormat=Query("webp"),
svc:TilesBuildService=Depends(get_tiles_build_service),
):
try:
returnsvc.build_for_uuid(uuid=uuid,tile_size=tile_size,fmt=fm
t,lossless=False)
exceptFileNotFoundError:
raiseHTTPException(status_code=404,detail="Original not foun
d")
exceptExceptionase:
raiseHTTPException(status_code=400,detail=f"Build failed:
{e}")
```

Теперь воркер “выполняет маршрут 2” (по сути), но без HTTP

В `tile_worker.py` в `handle_message()` вместо ручной логики будет:

```
fromapp.api.depsimportget_tiles_build_service

build_svc=get_tiles_build_service()
build_svc.build_for_uuid(uuid=uuid,tile_size=tile_size,fmt=fm
t,lossless=lossless)
```

✅ Всё. Теперь это прям буквально: “воркер выполняет то же, что и маршрут 2”, потому что маршрут 2 и воркер используют один и тот же сервис.

А как связать маршрут 1 → очередь автоматически?

Ты это уже хотел: “когда закончился маршрут 1, ставим задачу”.

Это мы и делаем “по флагу” в ingest:

- после успешного `meta = svc.ingest(...)`
- если `build_tiles=true` → `publish_build(payload)`

Если ты всё-таки хочешь, чтобы воркер реально дергал HTTP маршрут 2

Можно, но я не советую. Если всё равно хочешь — скажи, сделаю вариант:

- воркер делает `requests.post("http://api:28000/tiles/{uuid}/build?...")`
- надо будет добавить внутреннюю сеть, адрес `api`, таймауты, ретраи
- и аккуратно не словить циклы/нагрузку

Если ок, я прямо следующим сообщением дам готовый `tile_worker.py` вариант, который вызывает `TilesBuildService`, и покажу, какие импорты/depends поправить.